SYSTEM DESIGN

Foundational Concepts Explained with Real-World Examples

A practical beginner's guide

Learn how clients, APIs, servers, databases, caches, scaling, load balancers, queues, and service architectures work together.

The restaurant analogy

Imagine a busy restaurant. A customer places an order, a waiter carries the order to the kitchen, a chef prepares the food, and the restaurant stores ingredients and coordinates many orders. A software system follows a similar pattern: a client sends a request, an API defines how to ask for work, a server processes it, and databases and other components support the operation.

Includes: plain-language explanations • real-time scenarios • request flows • trade-offs • revision checklist

How to use this guide

Read the concepts in order. For each one, first learn what it means, then follow the restaurant analogy, and finally connect it to a familiar digital product such as an online shopping app, food-delivery app, streaming service, or banking app.

A simple system-design picture

```
User → Client app → API / network → Server → Database
```

A production system often adds a cache, load balancer, background workers, message queue, monitoring, and multiple services. These are not mandatory for every app; they are introduced when the product's scale, reliability, or complexity calls for them.

Contents

- 1. Client and Server
- 2. API (Application Programming Interface)
- 3. Frontend vs. Backend
- 4. Database
- 5. Cache, cache hit and cache miss
- 6. Scaling: vertical, horizontal and auto-scaling
- 7. Load Balancer
- 8. Synchronous vs. asynchronous communication
- 9. Stateful vs. stateless architecture
- 10. Monolith vs. microservices
- 11. Putting it together: food-delivery order
- 12. Design trade-offs and common mistakes
- 13. Glossary and revision questions

Important: system design is about trade-offs, not collecting as many technologies as possible. A small application may work best as a simple monolith with one database. Add complexity only when a real requirement justifies it.

1. Client and Server

What it means

The **client** is the application or device the user interacts with: a mobile app, browser, desktop application, or another program. It displays screens, collects input, performs some local validation, and sends requests over a network.

The server is a computer or group of computers that receives requests and performs work such as checking permissions, applying business rules, reading or updating data, and returning a result. A server can be a physical machine, a virtual machine, or a cloud-hosted service.

Restaurant analogy

At a restaurant, the customer is like the client: they decide what they want and communicate an order. The kitchen is like the server: it checks the order, prepares the meal, and returns the result. The analogy is not exact—software servers can serve many clients at once, and a client is not necessarily a human-facing app.

Real-world example

In a shopping app, the client displays a product page and lets the user tap “Add to cart.” The server checks that the item exists, confirms the current price and stock, and records the cart change. The client should not be trusted to decide the final price because users can tamper with client-side code.

Design notes and trade-offs

Client-side work makes the interface responsive; server-side work protects shared data and enforces rules. The client may temporarily show a loading indicator while the server processes a request. A network failure means the client might not receive a response even if the server did some work, so important operations often need safe retry handling.

2. API (Application Programming Interface)

What it means

An **API** is a defined way for software components to communicate. A web API commonly specifies endpoints or routes, request methods, accepted inputs, authentication requirements, response formats, and error behavior. It is not literally a waiter or necessarily a separate machine; it is the contract through which a client asks a service to do something.

Restaurant analogy

The waiter takes a customer's order in a known format, carries it to the kitchen, and brings back the result. The customer does not need to know the chef's recipe or kitchen layout. Likewise, a client can request a product list without knowing how the server queries its database.

Real-world example

A mobile app might send `GET /products/42` to retrieve product 42. The server could respond with JSON containing the name, price, and availability. A checkout action might use `POST /orders` with cart items and a delivery address. The server must validate all submitted values.

Design notes and trade-offs

Good APIs use clear names, predictable response structures, authentication and authorization where needed, validation, useful error codes, and versioning strategies when contracts change. Never place secret database credentials in a mobile app. An API is not automatically secure merely because it hides the implementation.

3. Frontend vs. Backend

What it means

The **frontend** is the user-facing part: screens, buttons, forms, navigation, and interactions. Technologies can include HTML, CSS, and JavaScript for web apps, Swift for iOS, and Kotlin for Android. The **backend** runs server-side logic such as account checks, pricing, payment coordination, notifications, and database access. Common backend languages include Java, JavaScript/Node.js, Python, PHP, Go, and others.

Restaurant analogy

The dining area and menu resemble the frontend: customers see them and use them to express their choices. The kitchen's preparation and coordination resemble the backend: most of that work is hidden from the customer but determines the result.

Real-world example

In a food-delivery app, the frontend displays restaurants, menus, order status, and a checkout screen. The backend calculates the total, checks whether the restaurant accepts orders, creates the order, coordinates payment, and stores the status. The frontend may show “Order placed,” but only the backend can reliably confirm that the order was accepted.

Design notes and trade-offs

Frontend and backend are roles, not a strict language split. JavaScript can run in both places. Some validation belongs in the frontend for a better experience, but the backend must repeat important validation because clients can be modified or bypassed.

4. Database

What it means

A **database** stores and retrieves application data. Relational databases such as PostgreSQL and MySQL organize data into tables and support relationships and transactions. NoSQL systems, such as document or key-value databases, use different data models and may fit particular access patterns. The right choice depends on data shape, consistency needs, query patterns, scale, and operational requirements.

Restaurant analogy

A restaurant's order book, customer records, and inventory list resemble database records. Staff consult them to find reservations, track ingredients, and update orders. A software database makes these operations durable and queryable.

Real-world example

An online store may keep customer accounts, products, inventory, orders, and payment references in a database. When a customer places an order, the server records the order and changes its status. For operations that must stay consistent—such as decrementing stock and creating an order—a database transaction or another carefully designed consistency mechanism may be needed.

Design notes and trade-offs

A database is not simply a folder of files. It provides structured operations, indexing, concurrency handling, and often transaction guarantees. Backups, access control, monitoring, migrations, and recovery tests are essential. A backup that has never been restored in a test is not a fully proven recovery plan.

5. Cache, cache hit and cache miss

What it means

A **cache** stores copies of data that are expensive or frequent to retrieve, usually in a faster layer such as application memory or Redis. The application checks the cache first. A **cache hit** means the requested item is present and usable; a **cache miss** means the app must fetch it from a slower source, often the database, then may store it in the cache for next time.

Restaurant analogy

A restaurant might keep the most popular menu items ready at the counter instead of checking the main storeroom for every question. That speeds up repeated requests, but the displayed information must be refreshed when prices or availability change.

Real-world example

A shopping app's popular product details might be cached for a short period. The first request misses the cache and reads the database. Later requests hit the cache and return faster. Inventory and prices require a deliberate freshness strategy so that a cache does not keep showing an item as available after it sells out.

Design notes and trade-offs

Caching reduces latency and database load, but adds invalidation and consistency challenges. Common approaches include time-to-live (TTL), explicit invalidation after updates, and cache-aside (the application loads from the database on a miss). Avoid caching sensitive user-specific data in a shared cache without correct keying and access controls.

6. Scaling: vertical, horizontal and auto-scaling

What it means

Scaling means adjusting capacity to handle more traffic or work. **Vertical scaling** increases the resources of one machine—for example, more CPU or RAM. **Horizontal scaling** adds more machines or instances and spreads work across them. **Auto-scaling** automatically increases or decreases the number of instances according to policies and metrics such as CPU use, request rate, or queue depth.

Restaurant analogy

Vertical scaling is like giving one kitchen a larger oven and more work surfaces. Horizontal scaling is like opening more kitchens and distributing orders among them. Auto-scaling is like opening another kitchen station when demand rises and closing it when the rush ends.

Real-world example

A ticket-booking website may handle ordinary traffic with a few servers but receive a large burst when popular tickets go on sale. Horizontal scaling can add application instances, and an auto-scaler can adjust capacity as demand changes. The database, payment provider, or another dependency can still become the bottleneck, so scaling only the web tier may not solve every problem.

Design notes and trade-offs

Vertical scaling is often simpler but has a practical upper limit and can leave one large machine as a failure point. Horizontal scaling can improve capacity and resilience but requires load distribution, health checks, and often stateless application instances. Auto-scaling has delay and cost trade-offs; it needs sensible minimums, maximums, and signals.

7. Load Balancer

What it means

A **load balancer** accepts incoming traffic and routes requests to available backend instances. It may distribute requests in round-robin order, by least connections, or using other policies. Health checks help it avoid sending traffic to unhealthy instances. Load balancers can operate at different network layers and may also terminate TLS or perform routing rules.

Restaurant analogy

A restaurant host directs incoming customers to available tables rather than sending everyone to one waiter. In a software system, the load balancer directs requests to healthy servers rather than letting one instance take all the traffic.

Real-world example

Suppose an online learning site runs four application instances. The load balancer routes each incoming request to an available instance. If one instance fails its health check, the balancer can stop sending new requests to it while the remaining instances continue serving traffic, provided they have enough capacity.

Design notes and trade-offs

A load balancer does not create extra capacity by itself; it distributes the capacity that exists. If all servers are overloaded or a shared database is down, a balancer cannot fix that. Session handling, sticky sessions, health-check accuracy, TLS configuration, and overload protection need consideration.

8. Synchronous vs. asynchronous communication

What it means

In **synchronous** request-response communication, the caller waits for a result before continuing that part of its work. In **asynchronous** communication, the caller can hand off work and continue while another component processes it later. A message queue or event stream—such as Kafka or RabbitMQ—can buffer work between producers and consumers. Asynchronous does not mean “instant” or “no waiting ever”; it means the work is decoupled from the immediate response path.

Restaurant analogy

For a made-to-order meal, a customer may wait while the chef prepares it (synchronous). For a catering request, the waiter might record the request and say the team will confirm later (asynchronous). The kitchen can process the request without keeping the customer interaction open.

Real-world example

When a user uploads a video, the app can store the upload and return “Processing started.” A background worker then generates thumbnails and multiple resolutions. The user can continue using the app. A queue helps absorb bursts and retry temporary failures, while the UI can poll for status or receive a notification when processing finishes.

Design notes and trade-offs

Queues introduce operational concerns: duplicate messages, retries, ordering, dead-letter handling, backpressure, and eventual consistency. Consumers should be idempotent where possible, meaning repeating the same message does not accidentally perform the action twice. Use synchronous calls when the user truly needs the result immediately; use background work for tasks that can safely finish later.

9. Stateful vs. stateless architecture

What it means

A **stateful** server keeps client-specific session information in its own memory or local storage between requests. A **stateless** application server does not depend on local session memory to handle the next request; necessary context is sent with the request or stored in a shared service. Stateless services can be easier to scale horizontally, but they still use stateful systems such as databases. “Stateless” does not mean the entire application has no state.

Restaurant analogy

If the same waiter must remember every detail of a customer's previous visit, the interaction resembles a stateful design. If the order ticket contains the information needed by any available staff member, it resembles a stateless request handled by any server.

Real-world example

A web service may authenticate requests with a signed access token, or look up a session in a shared session store. If any healthy server can validate the request and access shared data, requests can move between instances. JWT is one token format, not a requirement for stateless architecture; token expiration, revocation, and secure storage still matter.

Design notes and trade-offs

Stateful sessions can be straightforward but may require sticky routing or shared session storage when servers multiply. Stateless app servers ease failover, but tokens should not contain secrets and must be validated correctly. Choose the approach based on security, revocation needs, user experience, and operational complexity.

10. Monolithic vs. microservices

What it means

A **monolith** packages much or all of an application's backend into one deployable unit. It can still have well-separated internal modules. **Microservices** split a system into independently deployable services aligned to capabilities—for example, identity, catalog, orders, and notifications—with communication through APIs or events.

Restaurant analogy

One kitchen preparing every part of every meal resembles a monolith: coordination is centralized and simple, but one crowded kitchen can become a bottleneck. Several specialized stations resemble microservices: each station owns a narrower job, but coordination between stations becomes more important.

Real-world example

A small startup's booking app may begin as one backend containing login, listings, and bookings. As teams and workloads grow, the company might split notifications or search into separate services. That can allow independent scaling and deployment, but each network boundary introduces failure modes and monitoring needs.

Design notes and trade-offs

Microservices are not automatically faster, cheaper, or easier to debug. They add network calls, distributed tracing, deployment complexity, data ownership decisions, and cross-service consistency problems. A modular monolith is often a strong starting point. Split a service when there is a clear reason such as different scaling needs, team ownership, release independence, or fault isolation.

11. Putting it together: placing a food-delivery order

Imagine a user ordering dinner from a food-delivery app. The following is a simplified architecture; real companies vary in their implementation.

Request flow

1. The user selects a restaurant and taps “Place order.”
2. The mobile client sends an API request over HTTPS.
3. A load balancer routes the request to a healthy application instance.
4. The backend authenticates the user and validates the cart, price, address, and restaurant status.
5. The backend writes the order to the database using appropriate transaction boundaries.
6. The system requests payment authorization or coordinates payment with a payment provider.
7. The backend returns an order ID and current status to the client.
8. A message or event is published so background workers can notify the restaurant, send a confirmation, or update analytics.
9. The client refreshes or subscribes to status updates as the order moves through accepted, preparing, picked up, and delivered.

Illustrative architecture

```
Customer → Mobile app → Load balancer / API gateway → Order service → Database
■ Cache (for suitable frequently read data)
■ Message queue → Notification worker / Analytics worker
```

Where the concepts appear

Concept	Role in the example
Client / frontend	Shows restaurants, cart, checkout, and order status.
API	Defines how the app submits an order and reads its status.
Backend / server	Validates the request and applies ordering rules.
Database	Persists orders, users, and other durable records.
Cache	Speeds up suitable repeated reads, such as public restaurant details.
Load balancer	Routes incoming requests to healthy application instances.
Horizontal scaling	Adds application instances during high demand.
Asynchronous queue	Lets notifications and other slower tasks run outside the immediate checkout response.
Stateless app servers	Allow successive requests to be handled by different healthy instances when session design supports it.
Monolith or microservices	Describes how backend capabilities are packaged and deployed.

Failure scenarios to think about

- **The client times out after placing an order.** The user may retry. Use an idempotency key or another deduplication approach so a retry does not create duplicate orders.
- **A server instance crashes.** Health checks and the load balancer can route new requests elsewhere; shared durable data helps another instance continue.
- **The cache is stale.** Do not treat cached inventory as the final authority at checkout; verify critical values against the authoritative system.
- **The notification queue is delayed.** The order may already exist even if the email or push notification is late. Monitor queue lag and retry safely.

- **The payment result is uncertain.** Track payment state carefully and reconcile with the payment provider instead of blindly charging again.

The central lesson: a system design is a set of components plus the rules for how they behave when traffic rises, data changes, or something fails.

12. Design trade-offs and common mistakes

A good design starts from requirements. Before choosing components, ask: how many users and requests are expected, what data must never be lost, what latency is acceptable, what happens during a failure, and how much complexity can the team operate?

Common mistakes

- **Adding microservices too early.** A simple modular monolith is often easier to build, test, and deploy while the product is small.
- **Assuming a cache is always correct.** Cached data can be stale. Identify which values can tolerate delay and which must be checked against the source of truth.
- **Thinking a load balancer solves every performance issue.** It cannot fix a saturated database, slow external API, or inefficient query.
- **Making everything asynchronous.** Queues improve decoupling but make status, retries, and eventual consistency more complex.
- **Trusting the frontend.** Enforce authorization, price, stock, and other business rules on the server.
- **Confusing stateless servers with stateless systems.** Stateless application instances commonly rely on stateful databases and shared stores.
- **Ignoring observability.** Logs, metrics, traces, alerts, and dashboards help detect and diagnose problems.

A practical decision guide

If the problem is…	Consider…	Watch out for…
Repeated reads are slow	Caching, indexes, query optimization	Stale data and invalidation
One app instance is overloaded	Profiling, vertical scaling, then horizontal scaling if appropriate	Bottlenecks elsewhere
Traffic spikes unpredictably	Auto-scaling, queue buffering, rate limiting	Scale-up delay and cost
Slow noncritical work delays the response	Background jobs and a queue	Retries, duplicates, queue lag
One deployment is too risky or teams are blocked	Better module boundaries; possibly service extraction	Distributed-system complexity
Sessions break across servers	Shared session store or well-designed token approach	Security and revocation

13. Glossary

Term	Plain-language meaning
Latency	Time taken for a request or operation to complete.
Throughput	Amount of work completed per unit of time.
Availability	How often a system is usable when needed.
Reliability	How consistently the system behaves correctly over time.
Scalability	Ability to handle increased workload by adding or adjusting resources.
Endpoint	A particular API address or operation a client can call.
Authentication	Checking who a user or service is.
Authorization	Checking what that identity is allowed to do.
TTL	Time-to-live; how long a cached item is considered fresh before expiry.
Idempotency	Repeating an operation has the same intended effect as doing it once.
Message queue	A buffer that stores work messages until consumers process them.
Fault tolerance	Ability to keep providing acceptable service despite certain failures.
Observability	Using logs, metrics, traces, and related signals to understand system behavior.

Revision questions

- What is the difference between a client and a server?
- Why should a client use an API instead of connecting directly to a database?
- What is the difference between a cache hit and a cache miss?
- When might horizontal scaling be preferable to vertical scaling?
- What does a load balancer do, and what can it not fix?
- Give one suitable use case for asynchronous processing.
- Why can stateless application instances be easier to scale?
- Why might a monolith be a better starting point than microservices?
- What could go wrong if a checkout request is retried? How could idempotency help?

Suggested answers

1. The client initiates interactions; the server processes requests and provides results. 2. An API defines a controlled contract and keeps database credentials and internal implementation on the server. 3. A hit finds usable cached data; a miss fetches it from the source. 4. Horizontal scaling distributes work across instances and can add capacity beyond one machine. 5. A load balancer routes traffic; it cannot remove a database or dependency bottleneck. 6. Video processing, email delivery, report generation, or notifications. 7. Any healthy instance can handle a request without depending on local session memory. 8. A monolith is simpler to build and operate at small scale. 9. A retry can create duplicate orders or charges; idempotency keys and deduplication can make retries safe.

Final takeaway

System design is the practice of organizing software components so they meet real requirements for functionality, speed, scale, reliability, security, and cost. Start with the simplest architecture that satisfies the requirements, understand its bottlenecks, and introduce caches, queues, load balancers, scaling policies, or service boundaries when they solve a demonstrated problem.

This guide expands the concepts supplied in the conversation with general software-engineering explanations and illustrative examples. The examples are educational and do not describe the internal architecture of any specific company.